



P R O Y E C T O F I N A L

Torreta Inteligente

Controlada por Bluetooth y RC

con STM32 + Android + HC-05

Jorge Olaf Quijas Pérez
Jorge Emmanuel Roldán Márquez

Sistemas Embebidos • IBERO Ciudad de México • 2025

1. Descripción General del Sistema

El proyecto consiste en una torreta automatizada controlada de forma inalámbrica mediante Bluetooth. Un microcontrolador STM32 (Blue Pill) recibe comandos desde una aplicación Android desarrollada en Kotlin con Jetpack Compose. El sistema integra un servomotor para el apuntado horizontal, un motor DC 775 como mecanismo de disparo, y un sensor ultrasónico HC-SR04 para la medición de distancias en tiempo real.

La comunicación se establece a través del módulo HC-05 (UART a 9600 bps). El usuario puede orientar la torreta, activar el disparo y consultar la telemetría directamente desde su smartphone.

Componente	Función
STM32 Blue Pill	Controlador principal del sistema
HC-05 Bluetooth	Comunicación inalámbrica con Android
Servo SG90/MG996R	Movimiento horizontal de la torreta
Motor DC 775 + L298N	Mecanismo de disparo
HC-SR04	Medición de distancia ultrasónica

2. Diagrama de Conexión

La siguiente tabla describe el cableado completo entre los módulos y el STM32:

Motor DC 775 + Driver L298N	
Motor 775	Motor A (L298N)
Fuente 12V +	VMS (L298N)
Fuente GND	GND (L298N)
ENA	PA1 (STM32)
IN1	PA4 (STM32)
IN2	PA5 (STM32)
GND L298N	GND STM32
5V L298N	5V externo
Módulo Bluetooth HC-05	
VCC	5V
GND	GND
TX	PA10 (STM32 RX)

RX	PA9 (STM32 TX)
Sensor Ultrasónico HC-SR04	
VCC	5V
GND	GND
TRIG	PA2 (STM32)
ECHO	Divisor voltaje → PA3
Servomotor SG90 / MG996R	
Señal PWM	PA0 (STM32)
VCC	5V
GND	GND

Divisor de Voltaje — ECHO del HC-SR04

El pin ECHO del HC-SR04 opera a 5 V, mientras que el STM32 soporta únicamente 3.3 V en sus entradas digitales. Se usa un divisor resistivo de dos resistencias de 330 Ω para reducir la señal de forma segura:



3. Protocolo de Comandos Bluetooth

La aplicación Android envía caracteres ASCII de un byte al STM32 a través del HC-05 (UART, 9600 bps, 8N1). El microcontrolador interpreta cada comando y ejecuta la acción correspondiente:

Comando	Acción
L	Mover torreta a la izquierda
R	Mover torreta a la derecha
S	Activar disparo (~1 segundo)
M	Cambiar a modo manual
A	Cambiar a modo automático

La torreta también envía telemetría de vuelta a la app con el formato:

```
D:45 // Distancia medida en cm
"Sistema listo\r\n" // Mensaje de inicio
```

4. Código Documentado

4.1 Activación del Motor DC

Activa el motor DC a través del driver L298N configurando los pines IN1, IN2 y ENA del puerto A del STM32. IN1 = 1 e IN2 = 0 define la dirección de giro; ENA habilita la salida del driver.

```
void motor_on(void)
{
    GPIOA->ODR |= (1 << 4); // IN1 = 1 → sentido de giro definido
    GPIOA->ODR &= ~(1 << 5); // IN2 = 0 → dirección única
    GPIOA->ODR |= (1 << 1); // ENA = 1 → habilita el driver L298N
}

void motor_off(void)
{
    GPIOA->ODR &= ~(1 << 4); // IN1 = 0
    GPIOA->ODR &= ~(1 << 5); // IN2 = 0
    GPIOA->ODR &= ~(1 << 1); // ENA = 0 → motor detenido
}
```

4.2 Control de Posición del Servomotor (PWM)

El servomotor se controla mediante señal PWM generada por el Timer del STM32 en PA0. El pulso varía entre ~500 μ s (0°) y ~2500 μ s (180°). Las funciones precalculadas establecen posiciones angulares fijas para los extremos izquierdo (60°), centro (90°) y derecho (120°).

```
// Configura el ancho de pulso PWM en microsegundos
void setServoPulse(uint16_t pulse_us)
{
    TIM2->CCR1 = pulse_us; // Registro de comparación del Timer 2, canal 1
}

// Posición izquierda - 60°
void setServoAngleLeft(void)
{
    setServoPulse(1200); // 1 200  $\mu$ s → ~60°
    angulo_actual = 60;
}

// Posición central - 90°
void setServoAngleCenter(void)
{
    setServoPulse(1500); // 1 500  $\mu$ s → ~90° (neutro)
    angulo_actual = 90;
}

// Posición derecha - 120°
void setServoAngleRight(void)
```

```
{
    setServoPulse(1800);      // 1 800 µs → ~120°
    angulo_actual = 120;
}
```

4.3 Medición de Distancia Ultrasónica (HC-SR04)

Se genera un pulso de disparo de 10 µs en el pin TRIG (PA2). El sensor emite una ráfaga de ultrasonido y devuelve en ECHO (PA3) un pulso cuya duración es proporcional a la distancia. La distancia en cm se calcula dividiendo el tiempo de ida y vuelta entre el factor 58 (velocidad del sonido a temperatura ambiente).

```
uint32_t medir_distancia(void)
{
    // 1. Pulso de disparo: 10 µs en TRIG
    GPIOA->ODR |= (1 << 2);    // TRIG = HIGH
    delay_us(10);
    GPIOA->ODR &= ~(1 << 2);   // TRIG = LOW

    // 2. Esperar flanco de subida en ECHO
    while (!(GPIOA->IDR & (1 << 3)));
    uint32_t t_start = micros();

    // 3. Esperar flanco de bajada en ECHO
    while (GPIOA->IDR & (1 << 3));
    uint32_t t_end = micros();

    // 4. Distancia = tiempo / 58 (µs → cm)
    return (t_end - t_start) / 58;
}

// Llamada en el loop principal:
uint32_t distancia_cm = medir_distancia();
// Envío a la app: "D:45\r\n"
USART1_sendString("D:");
USART1_sendInt(distancia_cm);
USART1_sendString("\r\n");
```

4.4 Comunicación UART — Bluetooth HC-05

El STM32 usa USART1 (PA9 = TX, PA10 = RX) a 9600 bps para intercambiar datos con la app. La función de recepción se implementa por interrupción para no bloquear el loop de control.

```
// Envío de cadena de texto
void USART1_sendString(const char *str)
{
    while (*str) {
        while (!(USART1->SR & USART_SR_TXE)); // Esperar TX vacío
        USART1->DR = *str++;
    }
}

// ISR de recepción de comandos
void USART1_IRQHandler(void)
{
    // ...
```

```
if (USART1->SR & USART_SR_RXNE) {  
    char cmd = USART1->DR;  
    switch (cmd) {  
        case 'L': setServoAngleLeft(); break;  
        case 'R': setServoAngleRight(); break;  
        case 'S': disparar(); break;  
        case 'M': modo = MANUAL; break;  
        case 'A': modo = AUTOMATICO; break;  
    }  
}  
  
// Mensaje de inicio  
USART1_sendString("Sistema listo\r\n");
```

5. Fotografías del Prototipo

La estructura de la torreta fue construida con tubería PVC para el cañón y una base giratoria acoplada al servomotor. La rueda amarilla en el mecanismo de disparo corresponde al motor DC 775 que impulsa los proyectiles mediante fricción.



Figura 1 — Prototipo físico de la torreta inteligente

En la imagen se pueden identificar: (1) el tubo PVC que actúa como cañón, (2) la rueda del motor DC 775 que genera la fuerza de lanzamiento, (3) la base giratoria blanca conectada al servomotor, y (4) los cables de conexión hacia la tarjeta STM32 y el driver L298N ubicados fuera del encuadre.